Session-Level Inference Control for LLM Agents on Analytical Databases

Sanjay Mishra
Independent researcher
Raleigh, North Carolina, USA
sanmish4@icloud.com

ABSTRACT

An analytical agent can recover one person's value from aggregates that each look large enough for a per-query minimum group size. This paper audits the session. Before a query runs, the gateway rewrites it to the weighted set of entities behind each answer that depends on a protected column and refuses the query that would make one entity's value an exact linear combination of answers already returned. The guarantee is exact recovery of one identity on one static snapshot. It is not an interval guarantee, and a sum of two balances is not counted as either balance.

The audit does not cost the same as per-query suppression. On the finance sessions at k=3, suppression refuses 0.6% of benign queries and 7.0% of benign sessions; the linear audit refuses 3.3% and 26.8%, and it stops the scripted two- and three-query differencing attacks that suppression does not. A portfolio of at most 100 queries against 100 targets recovered every target from suppression and from nested-pair auditing, and recovered none from the linear audit (Wilson upper bound 3.7%). With no minimum group size, a sum of two customers matched the sum of the two balances in 100 of 100 fresh sessions and determined neither balance. At k=3 that query is refused. Differential privacy is a complement, not a substitute: it bounds every release and does not return exact answers. A rank test over the rationals agrees with the 10^{-9} floating-point test on the weights in these workloads and catches a disclosure that the floating-point test misses when multiplicities are near 10^9.

PVLDB Reference Format:
Sanjay Mishra. Session-Level Inference Control for LLM Agents on Analytical Databases. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/sanmish4ds/least-privilege-llm-db-agents.

1 INTRODUCTION

A reporting role will answer two averages that each cover dozens of customers. Subtracting them yields the one customer in the difference. Wang et al. name this failure V8, lack of compositional leakage control: policies that approve each query in isolation do not track what the answers reveal together [18]. Classical statistical databases called the same pattern a tracker [5] and tested it with linear algebra [3]. Those audits were not built for an agent that reads a refusal and rewrites the query, and a per-query minimum group size does not implement them.

This paper is a defense for that compositional channel. A gateway in front of the agent's SQL tool keeps, for each protected column, the weight vector of every admitted answer. A join contributes a weight equal to the number of rows an entity adds, which is the integer coefficient in the provenance polynomial of the aggregate [1, 7]. The gateway refuses the query that would put some entity's unit vector in the span. The decision is about one identity, one column, and one static snapshot. It does not promise that a value stays inside a wide interval, that a sum of several people stays hidden, or that a later update of the table is covered.

The contributions are narrower than a first measurement of warehouse controls.

(1) A session audit for exact linear disclosure of one protected value, including join weights and a rational rank test. Native Snowflake and BigQuery controls were not measured here, so this is not a claim about them.
(2) A finance-session measurement of the benign cost. At k=3 the linear audit refuses more benign queries and more benign sessions than per-query suppression. At k=10 the query-refusal rates coincide and the attacks prevented do not.
(3) A 100-target query portfolio, with no language model, on which suppression and nested-pair auditing fail and the linear audit admits no exact recovery. A two-customer sum, when answered, reveals the sum and not either customer.
(4) A differential-privacy baseline with a minimum group size, reported as complementary to the audit, and an account of which model played which role in the cached agent studies.

2 RELATED WORK

Table 1 places the closest lines. The distinction that matters for V8 is whether a decision may depend on answers already returned, and whether the thing protected is one numeric value or a budget of rows.

Wang et al. study data agents and list eight vulnerabilities [18]. V8 is not a single-query bug. Their wording is that policies may approve each query alone and still leak through a sequence of aggregates. This paper takes that channel as the object of the defense. SecureSQL measures how much text-to-SQL models leak when they are the interface [16]. SafeNLIDB trains an alignment layer for natural-language database interfaces and reports that rule filters and agent filters struggle with inference attacks [11]. Neither audits the linear span of SQL answers. PIMENTO grounds text, queries, and policies into a relational substrate so contextual integrity and

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: What each line decides on. *Session*: earlier answers can change the decision. *Exact*: an admitted numeric answer is exact. V8 is an attack surface, not a defense.

Line	Session	Exact value	Object
Per-query minimum group	no	yes	group size
Audit expert [3]	yes	yes	linear span
DPSQL+ [12]	accountant	no	DP and frequency
FactGate [19]	set budgets	yes	declared facts
SafeNLIDB [11]	alignment	yes	NL interface
V8 [18]	the gap	–	compositional leak
This audit	yes	yes	one identity

differential privacy can be enforced on unstructured text [15]; it is not a session audit of warehouse aggregates.

FactGate, whose first draft was titled SessionBound, turns an approved task into set-valued budgets over declared facts and accounts for them on one PostgreSQL snapshot [19]. The authors state that inference from answered or refused predicates is not bounded. Our guarantee is the complementary statement: exact linear disclosure of one numeric attribute is refused, and row-count budgets are not what we account.

QuerySnout and QueryCheetah search for attribute-inference attacks against query-based systems [4, 17]. The portfolio in Section 4.3 is much smaller. It includes the tracker patterns those lines of work made routine, plus greedy range splits and a mutation loop. It is not their evolutionary search, and it does not claim to have discovered a new query shape. The three-query overlap was put in the portfolio because nested-pair auditing is known to miss it [3].

Differential privacy replaces exact answers with noise [6] and has been engineered for SQL [8]. DPSQL+ adds a minimum-frequency rule, a static SQL validator, and a privacy accountant on top of user-level differential privacy [12]. Section 4.5 is not a run of that library. It is a Laplace mechanism whose noisy count must clear k before a noisy sum is released, with basic composition. The audit and that mechanism answer different questions.

The rewrite that recovers the entities behind an aggregate is why-provenance, specialized to a weighted sum [1, 2, 7]. We use the weights only to decide refusal. We do not return a provenance explanation to the agent.

Our earlier conversational risk scores track dialogue, not SQL answers [14]. A journal version of that work is under review [13]. The overlap is the lesson that a session score can confound length with risk, and the use of stateful guardrails as a baseline. The SQL benchmark, the entity-set audit, and the numbers in this paper are not in that work. The repository note calls the journal manuscript a TIFS submission; this revision was asked to record it as TDSC. The venue should be confirmed before camera-ready. No text or table from that manuscript is copied here.

3 THE AUDIT AND ITS SCOPE

A protected column may be aggregated and may not be returned raw. For an admitted aggregate the gateway drops the aggregate, keeps the predicates and the grouping, and runs that key-only query. Each output group becomes a vector w whose coordinate w_i is the number of rows entity i contributes. The answer is $w \cdot x$ for the unknown vector x of per-entity values. An average is the same fact once the group size is known.

Three tests run before the answer is returned. The cell test refuses a group whose support has fewer than k entities. The nested-pair test refuses a group that contains, or is contained in, an earlier group and differs in fewer than k entities. The linear test refuses a group that would put any unit vector e_i in the span of the admitted weights. Refused queries add nothing to the history.

An illustrative case, not a measured trial. Keys 1..7 and k=3. The sums on $\{1, 2, 3, 4\}$, $\{4, 5, 6, 7\}$, and $\{1..7\}$ have supports 4, 4, and 7. The only nestings differ by three keys, so the cell test and the nested-pair test allow all three. Adding the three weight vectors produces $2e_4$. The linear test refuses the query that completes that span.

Proposition 3.1. *Suppose every admitted aggregate on a protected column passed the linear test, and values are unbounded reals. Then for every entity i and every real vector x consistent with the admitted answers, there is an x' with the same answers and $x'_i \neq x_i$.*

Proof. The test refuses a vector precisely when some e_i lies in the span of the admitted weights. If it never refused, e_i is outside that span, so the solution set of $Wx = b$ contains a line along which coordinate i moves. □

The proof says nothing about values restricted to a box, about a sum of two or more entities, or about predicates at two different thresholds. Those are the limits in the last paragraph of this section.

The linear test shipped with the cached experiments is modified Gram–Schmidt in floating point, with a coordinate treated as determined when the projection diagonal exceeds $1 - 10^{-9}$. Weights in the audit are integers, so the same test can be row reduction over the rationals, with no tolerance. The rational test is the one used as the oracle in Section 4.3. On 400 random trials (unit-weight ranges and join weights from 1 to 50) the two tests never disagreed, and they did not disagree on the search. They disagree on a constructed pair of weights $(10^9, 10^9)$ and $(10^9, 10^9 - 1)$: those two equations determine both variables, the rational test says so, and the floating-point test reports neither. The 10^{-9} test is therefore not a proof for huge multiplicities. The workloads below do not use multiplicities of that size. Turning the rational test on is a flag (`exact_linear`); the cached finance rates were computed with the floating-point test and were recomputed for k=3 and k=10 in this revision.

Predicates on a protected column are a different unknown. The gateway relaxes each comparison to true, takes the entities that could match, and applies the same three tests to that candidate set. A count of one key against a threshold has a candidate set of size one and is refused. This blocks the probes we ran. It is not a proof for combinations of different thresholds: each threshold is its own indicator vector, and auditing them as one vector is a conservative heuristic [10]. Strict mode refuses any predicate on the protected column, so the allow-or-refuse sequence depends only on the query text and the non-protected columns [9]. We did not find a cheaper strict rule that keeps that property. Its benign cost is remeasured in Table 3.

What is out of scope, on purpose. The audit does not refuse a query merely because it shrinks the feasible interval of a bounded value. On the 200 scripted tracker sessions, the answers the linear audit admits leave every target's linear-program interval at the full clipping domain (median width 11000; 0 sessions narrowed). The

Table 2: Roles in the cached studies. The judge is the rank test, not a model.

Model id	Role	What was run
`gpt-4o-mini`	SQL agent; attacker	220 BIRD tasks; 50 targets × 5 defenses, 15 queries
`gpt-4o`	SQL agent; attacker	220 tasks; 25 targets × 4 defenses (strict not run)
`claude-sonnet-5-5`	SQL agent; attacker attempt	220 tasks; 1 target × 5 defenses, declined, 0 SQL
`qwen3:4b-instruct`	SQL agent only	55 tasks, local
none	rank-test oracle	every disclosure bit

attacker's midpoint is then no better than guessing the middle of the domain (11.5%, the same as the prior 11.5%). Interval privacy is a different mechanism. Collusion across identities, and values that change between queries, are also out of scope.

4 EVALUATION

4.1 Which model did what

No model is the judge. Disclosure in the adaptive study and in Section 4.3 is the rank test. Table 2 is the assignment in the cached runs. Dated API snapshots were not stored; the identifiers are the strings in the result files. `claude-sonnet-5-5` was called as an attacker on one target and five defenses. The API returned a refusal on the first turn of each episode, so that model sent no SQL and contributes no attack result. It was the SQL agent on the BIRD capability sample. `qwen3:4b-instruct` was a local SQL agent on 55 tasks and was not an attacker. This revision adds no paid model call.

4.2 Finance sessions: the cost is not the per-query cost

The finance workload is the seeded TPC-H-shaped database (1,500 customers) and the 400 benign finance sessions (4366 queries) plus the scripted attacks. Table 3 is recomputed in this revision from that generator. At k=3 the linear audit and the nested-pair audit refuse the same 3.3% of benign queries and 26.8% of benign sessions. Per-query suppression refuses 0.6% of queries and 7.0% of sessions, and it stops 1.0% of the two-query trackers. The linear audit stops all of them, and it stops the three-query attacks that nested pairs mostly miss (100.0% against 12.0%). Strict mode keeps that prevention and refuses 25.6% of benign queries, because the workload filters on balances.

At k=10 the query-refusal rate of suppression, nested pairs, and the linear audit is the same, 20.1%. Prevention is not the same: suppression stops 76.0% of two-query trackers and the linear audit stops all of them. "Same cost as a minimum group size" is therefore a statement about this workload at k=10, not a statement about k=3, and it is a statement about query refusals, not about which attacks survive.

4.3 A query portfolio, without a language model

The search uses a fresh in-memory table of 100 targets drawn from 1,500 customers with integer balances in $[-1000, 10000]$. For each defense and target it may issue 100 queries. The portfolio is fixed: one customer's sum, the sum of that customer and one other, a segment aggregate and the same aggregate with the target removed, three overlapping ranges, greedy bisection of the key range, and mutation of ranges that contain the target. A second fresh session runs threshold bisection. The rational rank test decides exact recovery. Figure 1 is the outcome.

Table 3: Finance sessions, recomputed. *Queries* and *Sessions* are benign refusals. *Two* and *Three* are scripted differencing attacks prevented.

		Queries	Sessions	Two	Three
k=3	Min. group	0.6%	7.0%	1.0%	3.0%
k=3	Nested pairs	3.3%	26.8%	100.0%	12.0%
k=3	Linear	3.3%	26.8%	100.0%	100.0%
k=3	Strict	25.6%	94.0%	100.0%	100.0%
k=10	Min. group	20.1%	89.5%	76.0%	82.0%
k=10	Nested pairs	20.1%	89.5%	100.0%	83.0%
k=10	Linear	20.1%	89.5%	100.0%	100.0%
k=10	Strict	37.3%	97.8%	100.0%	100.0%

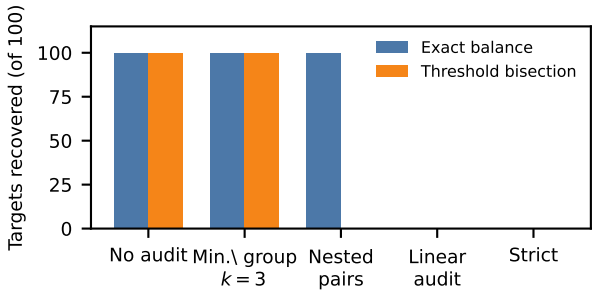


Figure 1: Targets recovered by the query portfolio. Exact balance: the unit vector is in the span of admitted sums. Bisection: a fresh session of threshold counts pins the integer balance. Nested pairs and the linear audit both run the predicate test, so bisection stops; minimum group size does not.

With no audit, the single-customer sum recovers 100 of 100. Minimum group size at k=3, 5, and 10 refuses that sum and still loses all 100 targets to the segment complement. Nested pairs refuse the complement and lose all 100 to the three-range overlap. The linear audit, the rational variant, and strict mode recover 0 of 100 (upper 95% Wilson bound 3.7%). The mutation loop did not recover any target that the named patterns had not already recovered. Calling this a QuerySnout-style discovery would overstate it: the winning patterns were written into the portfolio.

Bisection on a fresh session pins the balance for all 100 targets when the only rule is a minimum group size, and for none of them when the predicate test is on.

The two-customer sum needs its own session, because a search that has already isolated the target no longer shows what that one query reveals. With no minimum group size it was answered for 100 of 100 targets, the number matched the sum of the two balances in every admitted case, and neither customer's unit vector was in the span (0). Under the linear audit at k=3 the same query was refused for all 100 targets, because the group has size 2. It is a disclosure of a sum of two people. It is not a disclosure of either person.

Table 4: BIRD gold queries, one audit version (`bird_benign.json`). Min. group and linear refuse the same queries. Sessions is the share of the 200 sampled sessions with at least one refusal.

DB	n	Min. group	Linear	Strict	Sessions
fin	106	19.6%	19.6%	21.8%	88.5%
deb	64	41.5%	41.5%	41.5%	99.5%

Table 5: Execution accuracy for the columns-plus-keys grant. *All* divides by every task. *Admitted* divides only by queries the enforcer ran.

Model id	n	All	Admitted n	Admitted
`gpt-4o-mini`	220	40.5%	154	57.8%
`gpt-4o`	220	50.5%	197	56.3%
`claude-sonnet-5-5`	220	64.1%	202	69.8%
`qwen3:4b-instruct`	55	34.5%	45	42.2%

The cached adaptive attackers are a different experiment and a weaker budget. `gpt-4o-mini` (50 targets, 15 queries) and `gpt-4o` (25 targets, strict mode not run) both had zero exact disclosures under the linear audit, as decided by the floating-point oracle. Those episodes predate the predicate test. `claude-sonnet-5-5` is not part of that success rate. A local open-weight attacker with a budget of 100 queries was not run: no model server was listening, and this revision does not call a paid API. The harness is `experiments/purpose_sql/llm_attacker_local.py`.

4.4 Human-written queries, one audit version

BIRD's gold SQL is not written by the author of this paper. The committed audit log covers two databases, `financial` (106 queries) and `debit_card_specializing` (64 queries), under one version: weighted sums, the predicate test, k=3. Table 4 is that log. On both databases the per-query rule and the linear audit refuse the same share of queries. The session tests added no further refusals, which is why a "sessions" column matches a "rules" column. That is the measurement, not a copy-paste error. A second admitted-count split (93/41 against 90/40) is not in the repository and is not reported. Spider 2.0 runs are also not in the repository; there is no second Sonnet number to average.

Execution accuracy on the capability sample is the share of all tasks whose SQL ran and matched gold. Refused and invalid SQL count as wrong. Table 5 also shows the higher accuracy computed only on queries the enforcer admitted, for the columns-plus-keys grant. The published figure to use is the all-tasks column. The sample is 220 BIRD dev tasks (20 per database, seed 13), not the full dev set. The local model is a 55-task subset.

4.5 Differential privacy is complementary

Table 6 has two mechanisms on the finance sessions. The naive column is the cached Laplace baseline: one session budget, split evenly across protected aggregates, sensitivity 10,000 after clipping balances to $[-1000, 10000]$. The minimum-group column was run in this revision. Each query spends half of its share on a noisy count and half on a noisy sum, and the sum is released only when the noisy count is at least 3. Thresholding a noisy count is post-processing, so basic composition still applies. This is not an OpenDP or Tumult run, and it is not DPSQL+'s accountant [12].

Table 6: Finance sessions. Naive: cached Laplace, attacker within 10% of the balance. Min. group: noisy count must clear k=3; the last column is the share of draws whose two-query noise is within 10% of the balance, suppression counting as a miss.

ε	Naive err.	Naive att.	Suppress	Released err.	Diff. att.
1	87.7%	0.1%	25.2%	148.0%	0.1%
10	8.8%	2.2%	2.0%	17.3%	1.1%
100	0.9%	18.7%	0.0%	1.8%	11.2%

At ε=100 the naive attacker is within 10% of the target on 18.7% of tracker sessions, and the minimum-group mechanism's two-query difference is within 10% of the target's balance on 11.2% of draws, with 29 answers suppressed. At ε=1 the released answers have median relative error 148.0%. No budget here gives exact answers and a small attack success. The audit refuses 3.3% of benign finance queries and returns the rest unchanged, and it does not claim a privacy loss that composes over every possible query. The two belong together when a deployment needs both exact reports and a bound on queries the audit does not model.

4.6 Where the time goes

On the committed PostgreSQL replay, 112 of 2942 direct-path checks took more than one second. The maximum was 4.44 seconds, and the median check was 1.4 ms on the direct path and 1.8 ms through the proxy. A draft figure of 2.2 ms against 2.4 ms is not in this log. Medians of the pieces do not add to the median of the total. On the 2616 queries that executed, out of 2942, the median check, execute, and transport sum to 14.0 ms, while the median round trip is 53.7 ms. The round trip in the earlier table is a median over a different set of queries, including refusals that never execute.

Computing the entity set inside DuckDB, on one million customers, is a different measurement (this VM, not a warehouse). A `COUNT(DISTINCT)` over one nation (40,000 customers) took 3.0 ms median. The aggregated `(key, multiplicity)` result took 9.4 ms, and the same aggregation through a join to two million orders took 14.2 ms. Shipping the matching keys took 6.0 ms. One thousand successive range queries, width 5 to 100, grew the rational basis to rank 1000. The median check was 1.02 ms, the maximum was 3.71 ms, and 0 checks exceeded one second. Eight concurrent sessions of 50 queries on 200,000 customers each had a mean check of 4.6 ms. These ranges are not the heavy rewrites in the PostgreSQL tail. They show that the rank update is cheap once the database returns one row per entity.

5 THREATS TO VALIDITY

The finance sessions, the attack families, and the query portfolio were written by the same author as the audit. The portfolio confirms known gaps. It does not search the space QuerySnout searches, and the mutation step added no recovery. The adaptive language-model attackers used a budget of 15, were run before the predicate test, and do not include a model that agreed to attack: Sonnet declined, and no local model was available for a 100-query replacement.

BIRD accuracy is a 220-task sample, and the local model is 55 of those tasks. The benign audit log is two databases, not all of BIRD, and not Spider 2.0. Gold SQL that asks for one customer's amount is a refusal under an aggregate-only purpose; that rate is policy fit.

The floating-point rank test missed a constructed disclosure with weights near 10^9. The rational test is available and matched the floating-point test on the reported workloads. Interval narrowing, cross-threshold predicate combinations, and changes to the snapshot are outside the proof. Strict mode closes the predicate channel by refusing it, at the benign rate in Table 3.

The privacy numbers are basic composition of Laplace noise implemented in this repository. They are not a certificate from a DP library. The warehouse minimum-group measurements at k=3, 5, 10 were not run, because a Snowflake or BigQuery query can incur a charge. The SQL and the command are in `vendor_k_sweep.py`.

The DuckDB timings are one VM and one million synthetic customers. They do not replace the PostgreSQL tail, and they do not reproduce a 231-second check, which is not in the committed log.

6 CONCLUSION

The session audit refuses the query that would make one person's protected value an exact linear combination of answers already given. On the finance sessions that costs more benign refusals than a per-query minimum group size at k=3, and it stops a 100-target portfolio that defeats suppression and nested pairs. A sum of two customers is a sum, not either value. Differential privacy covers queries the audit does not model, and it does not return the exact answers the audit returns. The bound is exact recovery, per identity, on a static snapshot.

REFERENCES

[1] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2001. Why and Where: A Characterization of Data Provenance. In *Database Theory — ICDT 2001 (Lecture Notes in Computer Science)*. 316–330. https://doi.org/10.1007/3-540-44503-X_20

[2] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474. https://doi.org/10.1561/1900000006

[3] Francis Y. Chin and Gultekin Ozsoyoglu. 1982. Auditing and Inference Control in Statistical Databases. *IEEE Transactions on Software Engineering* SE-8, 6 (1982), 574–582. https://doi.org/10.1109/TSE.1982.236161

[4] Ana-Maria Cretu, Florimond Houssiau, Antoine Cully, and Yves-Alexandre de Montjoye. 2022. QuerySnout: Automating the Discovery of Attribute Inference Attacks against Query-Based Systems. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 623–637. https://doi.org/10.1145/3548606.3560581

[5] Dorothy E. Denning and Peter J. Denning. 1979. The Tracker: A Threat to Statistical Database Security. *ACM Transactions on Database Systems* 4, 1 (1979), 76–96. https://doi.org/10.1145/320064.320069

[6] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *Theory of Cryptography*. 265–284. https://doi.org/10.1007/11681878_14

[7] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. 2007. Provenance Semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. 31–40. https://doi.org/10.1145/1265530.1265535

[8] Noah Johnson, Joseph P. Near, and Dawn Song. 2018. Towards Practical Differential Privacy for SQL Queries. In *Proceedings of the VLDB Endowment*, Vol. 11. 526–539. https://doi.org/10.1145/3187009.3177733

[9] Krishnaram Kenthapadi, Nina Mishra, and Kobbi Nissim. 2005. Simulatable Auditing. In *Proceedings of the Twenty-Fourth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. 118–127. https://doi.org/10.1145/1065167.1065183

[10] Jon Kleinberg, Christos Papadimitriou, and Prabhakar Raghavan. 2003. Auditing Boolean Attributes. In *Journal of Computer and System Sciences*, Vol. 66. 244–253. https://doi.org/10.1016/S0022-0000(02)00036-3

[11] Ruiheng Liu, XiaoBing Chen, Jinyu Zhang, Qiongwen Zhang, Yu Zhang, and Bailong Yang. 2026. SafeNLIDB: A Privacy-Preserving Safety Alignment Framework for LLM-based Natural Language Database Interfaces. *arXiv preprint arXiv:2511.06778* (2026). AAAI 2026 extended version.

[12] Tomoya Matsumoto, Shokichi Takakura, Shun Takagi, and Satoshi Hasegawa. 2026. DPSQL+: A Differentially Private SQL Library with a Minimum Frequency Rule. *arXiv preprint arXiv:2602.22699* (2026). Accepted to TPDP 2026.

[13] Sanjay Mishra, Divya Chukkapalli, and Ganesh R. Naik. 2026. Stateful Guardrails for Multi-Turn LLM Systems. Journal version under review. The repository note names a TIFS manuscript; this revision was asked to record a TDSC submission. The author should confirm the venue.

[14] Sanjay Mishra, Divya Chukkapalli, and Ganesh R. Naik. 2026. Stateful Guardrails for Multi-Turn LLM Systems: A Conversational Risk Accumulation Framework. *arXiv preprint arXiv:2607.19361* (2026).

[15] Mushtari Sadia, Ang Chen, and Amrita Roy Chowdhury. 2026. PIMENTO: A Privacy Framework for Querying Text. *arXiv preprint arXiv:2609.14745* (2026).

[16] Yanqi Song, Ruiheng Liu, Shu Chen, Qianhao Ren, Yu Zhang, and Yongqi Yu. 2024. SecureSQL: Evaluating Data Leakage of Large Language Models as Natural Language Interfaces to Databases. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Miami, Florida, USA, 5975–5990. https://doi.org/10.18653/v1/2024.findings-emnlp.346

[17] Bozhidar Stevanoski, Ana-Maria Cretu, and Yves-Alexandre de Montjoye. 2024. QueryCheetah: Fast Automated Discovery of Attribute Inference Attacks against Query-Based Systems. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 3451–3465. https://doi.org/10.1145/3658644.3690272

[18] Kuncan Wang, Ziting Wang, Peizhuo Lv, Haoyang Li, Guoliang Li, Gao Cong, and Wei Dong. 2026. Data Agents Under Attack: Vulnerabilities in LLM-Driven Analytical Systems. *arXiv preprint arXiv:2606.08661* (2026).

[19] Minmin Wu. 2026. FactGate: A Task-Scoped Data Gateway for AI Agents with Cumulative Exposure Accounting. *arXiv preprint arXiv:2607.00751* (2026). Version 2. Version 1 was titled SessionBound.